

QuickDelivery - RabbitMQ

1. Objetivo

O RabbitMQ foi integrado ao QuickDelivery para demonstrar comunicação assíncrona entre o backend e um consumidor de eventos. O backend continua recebendo requisições REST, mas agora publica mensagens sempre que uma entrega é criada ou tem seu status alterado.

2. Por que RabbitMQ

O RabbitMQ foi escolhido porque é uma opção de MOM prevista no enunciado do projeto e possui suporte direto ao padrão de filas e exchanges. Ele também oferece painel web para visualização das filas, mensagens e consumidores, facilitando a demonstração da Sprint 2.

3. Estrutura Implementada

A integração usa uma exchange do tipo `topic` :

```
quickdelivery.events
```

A fila principal de eventos de entrega é:

```
quickdelivery.delivery-events
```

Eventos publicados:

- `delivery.created` : publicado após a criação de uma entrega.
- `delivery.accepted` : publicado após o aceite por um entregador.
- `delivery.status_changed` : publicado após qualquer mudança de status.

Fluxo da mensagem:

```
Backend REST -> Exchange quickdelivery.events -> Fila quickdelivery.delivery-events -> Consumer
```

4. Como a Integração Foi Feita

O RabbitMQ roda via Docker Compose junto com o PostgreSQL. O backend usa a biblioteca `amqplib` para abrir conexão AMQP, declarar a exchange `quickdelivery.events` e publicar mensagens com routing keys

específicas.

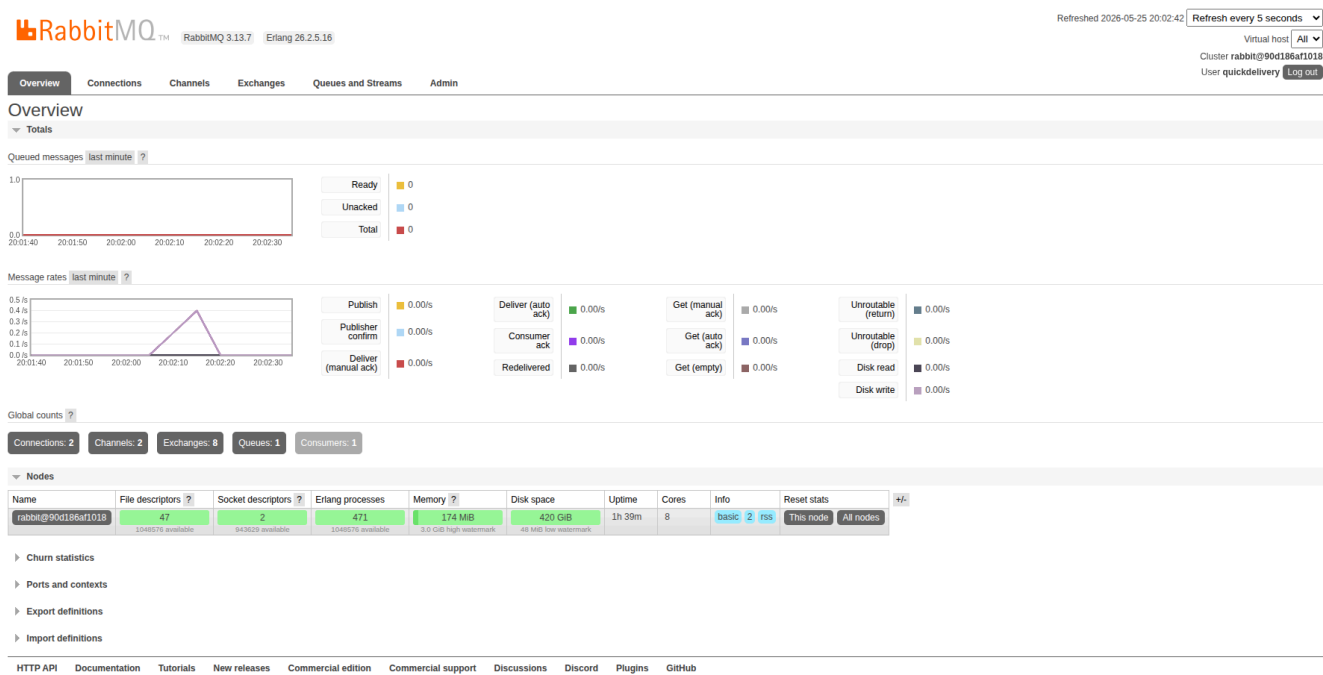
O arquivo `src/infrastructure/rabbitmq.ts` concentra a conexão com o RabbitMQ. O arquivo `src/events/delivery-events.publisher.ts` monta e publica os eventos de entrega. O arquivo `src/consumers/delivery-events.consumer.ts` executa um consumidor que escuta a fila `quickdelivery.delivery-events`.

O service de entregas publica eventos depois que a operação no banco é concluída com sucesso. Assim, o PostgreSQL continua sendo a fonte da verdade e o RabbitMQ fica responsável pela comunicação assíncrona.

5. Comprovantes

5.1 Consumer no RabbitMQ

O painel do RabbitMQ mostra a fila `quickdelivery.delivery-events` com um consumidor conectado.



Fila do RabbitMQ com consumer conectado

5.2 Logs do Consumer

O terminal do consumidor mostra os eventos recebidos após as ações executadas pelo Postman.

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

○ joaquimvilela@ONFNOTE128:~/puc/delivery-back\$ npm run consumer:deliveries

> quickdelivery-back@0.1.0 consumer:deliveries

> ts-node src/consumers/delivery-events.consumer.ts

[consumer] waiting for delivery events on queue "quickdelivery.delivery-events"

[consumer] delivery.created {

"eventId": "a232c70f-e93d-4895-9384-17be2ed93e46",

"eventName": "delivery.created",

"occurredAt": "2026-05-25T22:46:57.584Z",

"data": {

"deliveryId": "cdea070a-fa53-4fca-b48c-cd3d64305216",

"customerId": "d5febf42-7ca6-4082-a8bf-f39a22e3421a",

"pickupAddress": "Rua A, 100 - São Paulo",

"dropoffAddress": "Rua B, 200 - São Paulo",

"description": "Entrega de teste",

"status": "PENDING"

}

}

[consumer] delivery.status_changed {

"eventId": "569a01d5-e52f-4a28-af38-b6f620ceb224",

"eventName": "delivery.status_changed",

"occurredAt": "2026-05-25T22:49:53.213Z",

"data": {

"deliveryId": "cdea070a-fa53-4fca-b48c-cd3d64305216",

"customerId": "d5febf42-7ca6-4082-a8bf-f39a22e3421a",

"deliverymanId": "a6b5e25c-9a70-42ec-860d-e31ec997ba11",

"previousStatus": "PENDING",

"currentStatus": "ACCEPTED"

}

}

[consumer] delivery.accepted {

"eventId": "bebeea61-4a60-451b-a60d-ecd6d2cb7168",

"eventName": "delivery.accepted",

"occurredAt": "2026-05-25T22:49:53.227Z",

"data": {

"deliveryId": "cdea070a-fa53-4fca-b48c-cd3d64305216",

"customerId": "d5febf42-7ca6-4082-a8bf-f39a22e3421a",

"deliverymanId": "a6b5e25c-9a70-42ec-860d-e31ec997ba11",

"status": "ACCEPTED"

}

}

Logs do consumer recebendo eventos

Os logs comprovam que o backend publicou eventos no RabbitMQ e que outro processo consumiu essas mensagens pela fila, sem chamada REST direta entre eles.